

PDF Sherlock: An Offline, Privacy-Preserving Hybrid Retrieval Architecture for Academic Literature

Submitted May 2026, in partial fulfilment of
the conditions for the award of the degree BSc Computer Science.

20465963

School of Computer Science

University of Nottingham

I hereby declare that this dissertation is all my own work, except as indicated in
the text:

Signature: N.H.E.A

Date: 01/05/2026

I hereby declare that I have all necessary rights and consents to publicly
distribute this dissertation via the University of Nottingham's e-dissertation
archive.*

Public access to this dissertation is restricted until: 01/07/2026

Abstract

Modern information retrieval systems increasingly depend on dense neural embeddings, encouraging a shift toward Retrieval-Augmented Generation architectures hosted on centralised cloud infrastructure. This centralisation exposes sensitive academic and clinical documents to serious risks involving data sovereignty and requires expensive hardware, such as graphics processing units, to handle the high memory throughput needed by large vector indexes. At the same time, standard dense retrievers struggle with highly specialised technical vocabulary and often disrupt the semantic structure of long academic PDFs because they cannot parse complex multi-column layouts reliably. To overcome these challenges, this dissertation shows PDF Sherlock which is a fully offline, privacy-preserving hybrid retrieval architecture engineered specifically for academic literature running on standard central processing unit hardware. A document ingestion pipeline that is aware of page layout reconstructs multi-column texts using chunking aligned to text blocks, thus preserving semantic coherence. At query time, the architecture combines the precision of exact keyword matching from a sparse lexical index (BM25, implemented with Tantivy, which uses memory mapping) with the contextual recall of a dense semantic index (Sentence-Transformers, stored in a FAISS HNSW graph) through Reciprocal Rank Fusion. To bypass the local memory bottleneck, 8-bit Scalar Quantisation reduces the dense vector storage footprint by about 75 %. The system was thoroughly evaluated on a custom corpus of 156 academic PDFs containing over 6,600 chunks. The hybrid configuration achieves the highest Precision@10 (0.3000) and a Mean Reciprocal Rank (0.8667) that is statistically comparable to the strong BM25 baseline, while significantly outperforming the dense model alone (Wilcoxon signed-rank test, $p < 0.05$). It maintains a mean end-to-end query latency below 19 ms on both ARM-64 and x86-64 processors without any GPU acceleration. By decentralising these capabilities, PDF Sherlock delivers a secure, highly optimised, and cognitively accessible search tool for researchers and students.

Acknowledgements

I want to thank my supervisor Damla Kilic for her guidance and patience. She helped me stay on track when the work felt overwhelming and she never discouraged me from following the questions I cared about most.

To my mum and my sisters, thank you for your unconditional love and for always believing in me. There were times during this process when I struggled to feel understood, but you were my constant anchor. You quietly reminded me that true meaning isn't just a lexical property, it lives in the care, patience, and presence we give one another.

Information Retrieval, compression and quantisation have fascinated me for a long time. They are not just academic subjects. They touch something deeper about how we try to hold onto meaning despite noise and loss and distance. This dissertation is the result of that fascination, and of many late nights and long debugging sessions. I am grateful to have been able to bring it to the end.

Table of Contents

Abstract	i
Acknowledgements.....	ii
Chapter 1 Introduction.....	1
1.1. Motivation	3
1.2. Aims and Objectives.....	5
Chapter 2 Background and Related Work	6
2.1. Classical Information Retrieval - The Lexical Baseline	6
2.2. Neural Retrieval and Dense Embeddings	7
2.3. Efficient Vector Indexing and Hardware Constraints	8
2.4. The Need of Hybrid Search and Reciprocal Rank Fusion.....	9
2.5. Layout-Aware Document Parsing.....	9
2.6. Privacy, Edge Computing, and Centralised RAG	10
2.7. Existing Offline and Local Retrieval Systems.....	11
2.8. Summary of Research Gaps and Positioning.....	12
Chapter 3 Description of the Work	13
3.1 Overall System Intention.....	13
3.2 Functional Specification.....	13
3.3 Non-Functional Requirements.....	14
Chapter 4 Methodology	15
4.2. Evaluation Corpus and Ground Truth Construction	16
4.3. Experimental Variables and Configuration	17
4.3.1. Independent Variables (Retrieval Architectures)	18
4.3.2. Controlled Variables and Hardware Constraints	18
4.3.3. Experimental Configurations for RQ3 (Layout-Aware Ingestion).....	19
4.4. Evaluation Measures.....	20
4.5. Experimental Procedure	20
4.6. Statistical Techniques	21
4.7. Methodological Limitations	21
Chapter 5 Design	22

5.1. Architectural Overview	22
5.3 Hybrid IndexStore Design	24
5.5. User Interface Design	28
Chapter 6 Implementation.....	29
Chapter 7 Evaluation.....	32
7.1. Retrieval Effectiveness.....	32
7.2. Statistical Significance	34
7.3. Computational Efficiency and Hardware Constraints	34
Chapter 8 Summary and Reflections	35
8.1. Summary of Achievements	35
8.2 Reflection on Project Management	36
8.3. Intellectual Property and Research Ethics (LSEPI).....	36
8.4 Limitations.....	37
8.5. Future Work.....	38
Declaration of AI Usage.....	40
Bibliography.....	41

Chapter 1

Introduction

The primary objective of Information Retrieval (IR) systems is to parse, organise, and present relevant knowledge in response to user queries (Grainger, Turnbull and Irwin, 2024). Historically, text retrieval has been dominated by classical lexical approaches, most notably the BM25 probabilistic relevance framework (Robertson and Zaragoza, 2009, cited in Thakur et al., 2021). Lexical models represent queries and documents as high-dimensional, sparse vectors over a vocabulary, and calculate relevance using statistical associations such as term frequency and inverse document frequency (TF-IDF). Although these models are computationally efficient, interpretable, and robust for keyword-rich corpora, they are fundamentally constrained by their reliance on exact token matching. This constraint, referred to as the "lexical gap," prevents traditional IR systems from effectively capturing synonymy, concept drift, or semantic relationships when query terminology diverges from document text (Thakur et al., 2021).

The field of natural language processing (NLP) and IR underwent a significant transformation with the introduction of the Transformer neural network architecture in 2017 (Vaswani et al., 2017). By replacing traditional recurrence with self-attention mechanisms, Transformers enabled models to capture global dependencies between inputs and outputs, facilitating greater parallelisation and contextual understanding (Vaswani et al., 2017; Manning, 2022). This advancement initiated the era of Large Pretrained Language Models (LPLMs), such as BERT (Bidirectional Encoder Representations from Transformers) (Tunstall, von Werra and Wolf, 2022). Through self-supervised pretraining on extensive corpora, these models encode text into continuous, high-dimensional dense vector spaces (Manning, 2022). Within this vector space, semantic similarity is measured using linear algebraic operations, such as cosine similarity or dot product, enabling retrieval of documents based on underlying meaning rather than surface-level keyword overlap (Grainger, Turnbull and Irwin, 2024).

Although neural retrieval provides superior semantic understanding, it incurs significant computational costs. A straightforward dense retrieval approach requires calculating the similarity between a query vector and every document

vector in the corpus, resulting in an $O(N)$ linear scan that is impractically slow for large collections on standard central processing units (CPUs). Additionally, uncompressed high-dimensional floating-point vectors require substantial memory bandwidth, creating a "memory wall" that often necessitates the use of costly Graphics Processing Unit (GPU) acceleration for low-latency inference (Johnson, Douze and Jégou, 2017). As a result, modern retrieval systems and emerging paradigms such as Retrieval-Augmented Generation (RAG), which grounds language model outputs in external knowledge to reduce factual hallucinations, are increasingly centralised on cloud infrastructure (Chakraborty, Dahal and Gupta, 2025). This centralisation introduces significant data privacy and scalability challenges (Xu et al., 2025). In privacy-sensitive fields such as clinical healthcare or proprietary academic research, transmitting confidential data to external cloud servers or centralised knowledge bases shows substantial security and regulatory risks (Chakraborty, Dahal and Gupta, 2025; Xu et al., 2025). Consequently, there is a critical need to decentralise these architectures and deploy high-performance retrieval systems locally on edge devices and commodity hardware (Xu et al., 2025).

Long academic PDFs introduce an additional challenge that is under-represented in standard IR benchmarks. Collections such as MS MARCO and BEIR predominantly contain short passages, whereas theses and research articles are multi-page, hierarchically structured documents with sections, figures and equations. Chunking strategies, metadata design and layout-aware ingestion all affect how well dense and hybrid retrievers can surface relevant information at page or paragraph granularity, yet these design choices are rarely evaluated systematically (Grainger, Turnbull and Irwin, 2024; Pang et al., 2022). As argued in AI-powered and transformer-based treatments of semantic search, effective systems must integrate content understanding, user understanding and domain knowledge, rather than relying on any single component in isolation (Grainger, Turnbull and Irwin, 2024; Tunstall, von Werra and Wolf, 2022).

Furthermore, extensive zero-shot evaluations of retrieval models indicate that no single retrieval paradigm is universally optimal (Thakur et al., 2021). Dense retrievers perform well within their training domains but often experience significant performance declines on out-of-distribution data, where classical models like BM25 remain robust baselines. These findings underscore the need for hybrid retrieval architectures that combine the exact-match precision of sparse lexical models with the deep semantic recall of dense neural models (Grainger, Turnbull and Irwin, 2024).

To address the combined challenges of semantic accuracy, data privacy, and hardware limitations, this dissertation introduces PDF Sherlock which is a fully offline, privacy-preserving hybrid retrieval architecture designed for latency-constrained academic search. PDF Sherlock operates on standard commodity hardware by replacing exhaustive $O(N)$ vector comparisons with Hierarchical Navigable Small World (HNSW) graph indexing, which enables logarithmic $O(\log N)$ approximate nearest neighbor (ANN) search speeds. To mitigate the memory bandwidth bottleneck of dense embeddings, the system explores the use of 8-bit Scalar Quantisation (SQ8), reducing vector footprints by up to 75% while aiming to maintain relative ranking accuracy (Johnson, Douze and Jégou, 2017). By integrating this optimised semantic backend with BM25 lexical scoring and a layout-aware ingestion pipeline for longer academic literature, PDF Sherlock expands access to advanced AI-powered search, offering a secure and cognitively accessible tool for researchers and neurodivergent students.

This dissertation is organised into eight chapters. Chapter 2 provides the theoretical background and reviews related work in classical IR, neural retrieval, efficient vector indexing, and the development of RAG systems. Chapter 3 outlines the project specifications and functional requirements. Chapter 4 details the research methodology, outlining the research questions, evaluation corpus, and experimental variables. Chapter 5 describes the overall system design and architectural diagrams. Chapter 6 details the software implementation, including layout-aware data ingestion, the hybrid IndexStore, and the local service layer. Chapter 7 offers a comprehensive quantitative evaluation of the system, benchmarking retrieval quality (NDCG@10), query latency, and memory footprint across different architectural configurations. Finally, Chapter 8 concludes with a summary of contributions, a reflection on project management, and suggestions for future research.

1.1. Motivation

The motivation for developing PDF Sherlock stems from three intersecting challenges in the current landscape of information retrieval which are the privacy and hardware limitations of centralised neural search, the contextual complexity of academic literature, and the demand for cognitively accessible research tools.

First, as Large Language Models (LLMs) are increasingly integrated with external knowledge bases to reduce factual hallucinations which is a framework known as Retrieval-Augmented Generation (RAG), there is a growing reliance on centralised

cloud servers (Chakraborty, Dahal and Gupta, 2025). In domains such as clinical healthcare, proprietary engineering, and academic research, sending sensitive documents to external APIs violates strict data privacy regulations and confidentiality agreements. While emerging frameworks like Distributed RAG attempt to mitigate these risks by decentralising knowledge across peer-to-peer networks (Xu et al., 2025), the fundamental requirement for local, on-device execution remains. Uncompressed dense vector indices require massive Random Access Memory (RAM) bandwidth, effectively restricting state-of-the-art neural search to expensive, GPU-accelerated workstations (Johnson, Douze and Jégou, 2017). There is a critical, unmet need to democratise these capabilities by aggressively optimising AI models via vector quantisation, enabling them to run securely and locally on standard commodity CPUs.

Second, existing neural IR models have predominantly been studied and benchmarked in narrow, homogeneous settings focusing on short, web-centric passages, such as the MS MARCO dataset (Thakur et al., 2021). Academic PDFs, however, diverge significantly from these benchmarks. They are lengthy, hierarchically organised documents replete with irregular formatting, complex nomenclature, tables, and multi-page logic. Standard dense encoders possess fixed input token limits (typically 512 tokens), meaning they cannot ingest entire papers at once. Naive chunking approaches often sever vital semantic context across document sections (Grainger, Turnbull and Irwin, 2024). Furthermore, dense retrievers frequently struggle with out-of-distribution data and highly specialised technical vocabulary, where exact-match precision is important (Thakur et al., 2021). Therefore, a hybrid retrieval approach that fuses BM25's syntactic keyword matching with dense semantic representations is uniquely motivated to solve the specific complexities of the academic domain.

Beyond algorithmic constraints, traditional search mechanisms impose heavy cognitive loads on users. Standard boolean or exact-match keyword searching requires the user to proactively guess the exact terminology used by the author. For students and researchers dealing with neurodivergent conditions or attention difficulties (such as ADHD), holding complex concepts in working memory while repeatedly refining search queries creates severe "search friction". By allowing users to query local libraries using natural language and instantly navigating them to the exact relevant chunk within a complex PDF, PDF Sherlock is motivated by a social imperative to create equitable, cognitively accessible educational tools.

1.2. Aims and Objectives

The aims of this dissertation are to design, implement, and quantitatively evaluate PDF Sherlock, an offline, privacy-preserving hybrid retrieval architecture tailored for academic PDFs. The system aims to deliver the semantic intelligence of neural search within the strict memory and latency budgets of standard commodity CPU hardware, thereby assisting researchers and neurodivergent students in seamlessly navigating complex literature. To achieve this, here are some specific objectives.

1. **Hybrid Pipeline Implementation:** Develop a search engine that fuses sparse lexical signals (BM25) with dense vector embeddings (Sentence-Transformers), utilising Reciprocal Rank Fusion (RRF) to balance keyword precision with semantic recall.
2. **Hardware Optimization:** Design a latency-optimised backend utilising Hierarchical Navigable Small World (HNSW) graph indexing to achieve logarithmic $O(\log N)$ approximate nearest neighbour search speeds without GPU acceleration.
3. **Memory Footprint Compression:** Investigate and implement 8-bit Scalar Quantisation (SQ8) to compress 32-bit floating-point vector indices, aiming to reduce RAM requirements by up to 75% while preserving relative ranking accuracy.
4. **Layout-Aware Document Ingestion:** Engineer a robust ingestion pipeline using PyMuPDF that intelligently chunks multi-column, hierarchically structured academic PDFs, preserving semantic boundaries and spatial metadata such as page numbers, bounding boxes.
5. **Data Sovereignty and Interface Design:** Prototype a stateless FastAPI service and decoupled Streamlit frontend that execute entirely on the host machine, guaranteeing that no user queries or sensitive academic data are transmitted to external servers.
6. **Rigorous Quantitative Evaluation:** Construct an evaluation script to benchmark the hybrid system against sparse-only and dense-only baselines. Retrieval quality will be assessed using Normalised Discounted Cumulative Gain (NDCG@10) and Recall@k, with performance differences validated for statistical significance using the Wilcoxon Signed-Rank Test ($p < 0.05$).

Chapter 2

Background and Related Work

This chapter establishes the theoretical foundations and reviews the existing literature pertinent to the development of PDF Sherlock.

2.1. Classical Information Retrieval - The Lexical Baseline

The fundamental architecture of classical Information Retrieval (IR) relies on lexical, exact-match paradigms to surface relevant documents from a corpus. At the core of these systems is the "bag of words" hypothesis, which postulates that the relevance of a document to a query can be approximated by the overlapping frequency of terms, entirely disregarding the sequential order or syntactic structure of the language (Manning, Raghavan and Schütze, 2008). To achieve high-speed retrieval, classical systems rely on an inverted index. Rather than scanning documents sequentially, an inverted index operates as a highly optimised dictionary mapping every unique term in the corpus's vocabulary to a postings list, an array of document IDs indicating where that term occurs. This reduces the search process from a linear $O(N)$ scan to an $O(1)$ hash lookup, enabling the extreme computational efficiency that makes classical IR highly suitable for commodity hardware. The statistical foundation for scoring these matches is the Term Frequency-Inverse Document Frequency (TF-IDF) framework. TF-IDF assumes that a word's importance is proportional to how frequently it appears in a specific document (TF), but inversely proportional to how frequently it appears across the entire corpus (IDF). Thus, rare words such as "quantisation" carry more informational weight than common stop words such as "the". The state-of-the-art evolution of the TF-IDF framework is the Okapi BM25 probabilistic relevance model (Robertson and Zaragoza, 2009). BM25 remains the industry-standard baseline for sparse retrieval because it introduces two critical hyperparameters to solve the flaws of raw TF-IDF, term frequency saturation k_1 and document length normalisation (b). The scoring function for a document D and a query Q containing terms q_1 is defined as:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

Where $f(q_i, D)$ represents term frequency, $|D|$ is the document length, avgdl is the average corpus document length, k_1 is the saturation hyperparameter, and b is the length normalization factor.

Despite its mathematical elegance and extreme hardware efficiency, BM25 is fundamentally limited by the "lexical gap" (also known as the vocabulary mismatch problem). Because BM25 relies on discrete, exact token matching, a query for "neural networks" will completely fail to retrieve a highly relevant document that exclusively uses the synonymous term "deep learning" (Grainger, Turnbull and Irwin, 2024). Furthermore, BM25 cannot understand polysemy or contextual nuance. While recent zero-shot evaluations, such as the BEIR benchmark, show that BM25 remains remarkably robust on out-of-distribution data where modern AI models often hallucinate (Thakur et al., 2021), its inability to capture semantic meaning creates a rigid ceiling on retrieval recall. To bridge this gap, the field of IR required a paradigm shift from counting discrete words to understanding continuous meaning.

2.2. Neural Retrieval and Dense Embeddings

The shift toward neural retrieval was largely enabled by the Transformer architecture, which replaced recurrence with self-attention and thereby enabled models to capture global contextual dependencies across text (Vaswani et al., 2017). Foundation models such as BERT later exploited this capability to map language into continuous, high-dimensional dense vector spaces, where semantic similarity is measured by geometric proximity rather than by exact lexical overlap (Manning, 2022; Tunstall, von Werra and Wolf, 2022). As a result, a retrieval engine can match a query like "neural networks" with a document that only mentions "deep learning," because the two expressions land close together in the vector space (Grainger, Turnbull and Irwin, 2024). In practice, neural retrieval models are deployed in one of two architectural forms: cross-encoders and bi-encoders. Cross-encoders process the query and the document jointly, which yields highly accurate relevance scores but is computationally prohibitive at query time and cannot be pre-computed, they are therefore usually reserved for re-ranking a small set of top candidates (Grainger, Turnbull and Irwin, 2024). Bi-encoders, by contrast, produce independent embeddings for queries and documents. This decoupling is what makes scalable retrieval possible: the entire corpus can be embedded offline, and at query time only the short user query passes through the network, allowing high-volume matching via efficient distance metrics such as cosine similarity (Grainger, Turnbull and Irwin, 2024). Because PDF

Sherlock must observe strict latency constraints, it adopts a bi-encoder architecture. Despite their semantic power, dense models bring two significant difficulties to the academic domain. First, self-attention scales quadratically with sequence length, forcing standard pretrained models to impose a fixed token limit (typically 512 tokens). This makes them incapable of digesting a complete multi-page academic paper in one pass and necessitates careful chunking strategies. Second, dense retrievers often falter on out-of-distribution (OOD) data. The BEIR benchmark, a heterogeneous zero-shot evaluation framework, has shown that dense models regularly underperform classical sparse baselines such as BM25 when faced with the highly specialised technical vocabulary that characterises much scholarly writing (Thakur et al., 2021). That vulnerability is a major motivation for hybrid architectures that combine neural recall with the exact-match precision of lexical methods.

2.3. Efficient Vector Indexing and Hardware Constraints

While dense neural embeddings resolve the lexical gap, they introduce a substantial computational bottleneck at retrieval time. A naïve exact-match approach must compute a distance metric, typically Cosine Similarity, between the query vector and every document vector in the corpus, resulting in linear time complexity $O(N \times D)$. On standard CPUs, without the parallelism of a GPU, this exhaustive scan quickly becomes impractical as the collection grows. To achieve acceptable latency on commodity hardware, modern retrieval engines rely on Approximate Nearest Neighbour (ANN) algorithms. PDF Sherlock implements the Hierarchical Navigable Small World (HNSW) algorithm through the FAISS library. HNSW organises vectors into a multi-layered proximity graph, at query time the search greedily navigates from the top sparse layers down to the denser base layers, reducing the complexity from a linear $O(N)$ scan to a logarithmic $O(\log N)$ traversal. Even with fast graph-based search, dense retrieval encounters a second bottleneck in the form of a “memory wall.” Standard 384-dimensional float32 embeddings demand considerable RAM, and if an index exceeds the available memory the operating system falls back to disk swapping, at which point latency deteriorates sharply. To keep the system within the memory budget of a standard laptop, PDF Sherlock applies 8-bit Scalar Quantisation (SQ8). This lossy compression algorithm maps each float32 dimension onto one of 256 discrete integer values, theoretically shrinking the dense index footprint by 75%. While the absolute precision of the vector positions is slightly reduced, their relative ordering, and

therefore the ranking of semantic similarity, is largely preserved, making accurate dense search feasible on consumer hardware.

2.4. The Need of Hybrid Search and Reciprocal Rank Fusion

Lexical models such as BM25 offer high precision for exact-match keywords and rare terms but cannot capture semantic relationships, while dense retrievers understand synonymy and general intent yet often struggle with the specialised vocabulary of academic texts (Thakur et al., 2021). In a domain where a query might mix a broad concept with a specific acronym, neither paradigm is sufficient alone. A hybrid architecture that runs sparse and dense searches in parallel and fuses their outputs is therefore necessary (Grainger, Turnbull and Irwin, 2025).

The difficulty is that the raw scores from the two pipelines are mathematically incompatible: BM25 produces unbounded positive values, whereas dense models output bounded cosine similarities in $[-1,1]$, making direct averaging invalid. To avoid complex score normalisation, modern hybrid systems employ Reciprocal Rank Fusion (RRF), which uses only the ordinal ranks of documents, not their scores.

For a given document d appearing in multiple ranked lists R , the RRF score is calculated as:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

Where $r(d)$ is the rank of the document in a specific list, and k is a smoothing constant (typically set to 60) designed to mitigate the outsized impact of highly ranked outliers. RRF is evaluated as highly effective because it does not require complex parameter tuning or machine learning models to weight the pipelines. It operates on the intuitive premise that if a document is ranked highly by both the lexical keyword matching and the semantic neural understanding, it is statistically highly likely to be the most relevant result for the user.

2.5. Layout-Aware Document Parsing

Academic literature shows unique pre-retrieval challenges that are largely absent from standard IR benchmarks like MS MARCO, which focus on short, sequential web passages. Research articles, by contrast, are complex, multi-page documents with double-column layouts, mathematical equations, and structural noise such as recurring headers and footers. Conventional extraction libraries typically read

PDFs in a simple left to right, top to bottom order, in a multi-column paper this naïve pattern can merge unrelated text from separate columns, destroying semantic coherence. The strict token limits of Transformer models add a further complication, requiring long PDFs to be divided into smaller chunks before embedding. Naïve chunking strategies that rely on fixed word counts frequently sever paragraphs and scientific definitions across separate vectors, leading to significant context loss. To address this, PDF Sherlock employs a layout-aware pipeline powered by PyMuPDF. By analysing the geometric bounding boxes of text blocks, the system reconstructs the intended reading order and segments documents along natural semantic boundaries such as paragraphs. Preserving spatial metadata, page numbers and coordinates, with each chunk is essential for the user interface, as it enables the system to navigate directly to the correct passage and highlight it within the source document.

2.6. Privacy, Edge Computing, and Centralised RAG

The integration of retrieval engines with generative AI has resulted in the Retrieval-Augmented Generation (RAG) paradigm. RAG mitigates the hallucinatory tendencies of Large Language Models by anchoring their generative capabilities to a retrieved context window of verified external documents. However, the immense computational overhead of both the dense retrieval indexing and the LLM inference has driven the industry toward highly centralised, cloud-hosted architectures (Chakraborty, Dahal and Gupta, 2025).

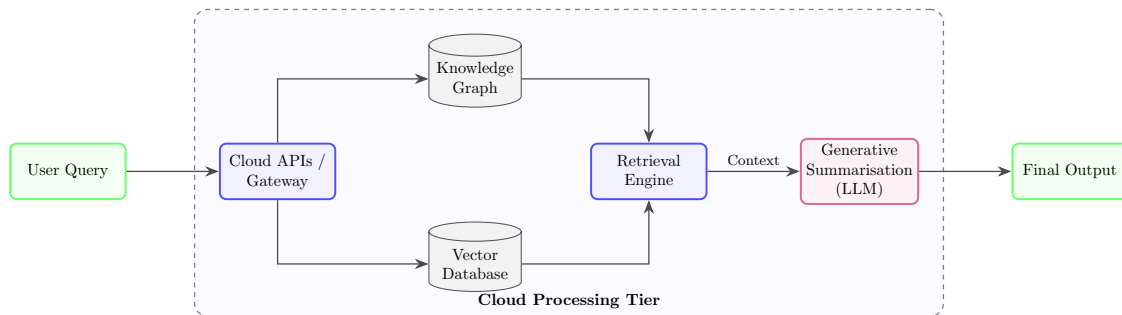


Figure 2.1: An example of a highly centralised, multi-tiered modern AI search architecture. Systems of this scale rely heavily on cloud processing, creating critical data sovereignty and privacy challenges for sensitive academic and clinical workflows.

As illustrated by the complex, multi-tiered architectures of modern commercial search systems, cloud reliance introduces an unavoidable "privacy gap." In domains heavily regulated by confidentiality agreements, such as clinical healthcare data, proprietary corporate engineering, or unreleased academic theses, transmitting sensitive documents to third-party cloud APIs constitutes a severe security violation (Chakraborty, Dahal and Gupta, 2025).

To resolve this tension between AI capabilities and data sovereignty, there is an emerging research focus on Distributed RAG and Edge AI, which seeks to decentralise retrieval and inference across local hardware networks (Xu et al., 2025).

2.7. Existing Offline and Local Retrieval Systems

Growing interest in data sovereignty has prompted the development of several open-source tools for private, local document search. General-purpose platforms such as PrivateGPT (Zylon-ai, 2025) and LocalGPT (PromptEngineer, 2025) show that transformer-based embeddings and hybrid search can run entirely offline. More recent entries like AnythingLLM (Mintplex-Labs, 2025) and WeKnora (Tencent, 2025) extend this further, offering full-featured RAG capabilities on local hardware with broad format support. These systems are, however, built for general use and do not account for the multi-column layouts or complex hierarchical structures typical of academic PDFs.

Tools with a specific academic focus, including Academic-Paper-RAG and ssearch, address some of these concerns by employing domain-specific embeddings such as SPECTER and preserving structural elements such as LaTeX notation. While they perform well on semantic recall, they generally do not systematically integrate lexical models like BM25, whose exact-match precision can be decisive when dealing with the specialised technical vocabulary of scholarly writing. Research prototypes such as Astrino provide encouraging evidence for the efficacy of hybrid retrieval, but often depend on GPU acceleration, which limits their practicality on standard commodity CPUs. Architecturally, the Offline MultiModal RAG System comes closest to PDF Sherlock, employing a FAISS or BM25 or RRF stack, however, it lacks layout-aware ingestion and the kind of rigorous statistical validation needed for academic corpora.

Positioning of PDF Sherlock

Unlike the systems reviewed above, which embed retrieval within larger generative frameworks, PDF Sherlock confines itself to the retrieval layer alone. This narrower scope reflects a deliberate choice: retrieval quality determines everything that follows, and even a strong language model cannot compensate for poorly ranked or irrelevant chunks. A retrieval-only architecture also permits objective evaluation using established metrics such as NDCG@10 and Recall@k, together with statistical tests like the Wilcoxon Signed-Rank Test, which are far harder to apply to subjective generative outputs. PDF Sherlock therefore fills a specific gap, a thoroughly evaluated, privacy-preserving hybrid retrieval engine that has been optimised for the structural complexity of academic literature and the tight memory and latency budgets of commodity CPU hardware.

2.8. Summary of Research Gaps and Positioning

The review identifies four connected gaps which are the lexical limitation of efficient models like BM25, the prohibitive hardware demands of dense retrieval, the structural complexity of academic PDFs overlooked by standard benchmarks and the privacy risks of centralised RAG architectures. Existing local tools show viability, but none simultaneously provides layout-aware ingestion, hybrid sparse-dense fusion, aggressive memory optimisation for CPUs, and rigorous statistical evaluation. PDF Sherlock addresses these together as it combines BM25 and HNSW-indexed dense retrieval through Reciprocal Rank Fusion, applies 8-bit scalar quantisation to run efficiently on commodity hardware, and uses a layout-aware pipeline that preserves document structure. Its retrieval-only design enables objective evaluation with NDCG@10 and the Wilcoxon Signed-Rank Test, delivering a focused, privacy-preserving foundation for academic literature search.

Chapter 3

Description of the Work

This chapter outlines the functional specification of PDF Sherlock, defining what the system is engineered to achieve and the specific operational workflows it supports.

3.1 Overall System Intention

The primary intention of PDF Sherlock is to provide an accessible, fully offline hybrid search engine tailored specifically for longer academic literature. It is designed to solve the "search friction" experienced by researchers and neurodivergent students who struggle to navigate complex PDFs using traditional exact-match keyword tools (Ctrl+F). The system functions as a privacy-preserving desktop application that runs entirely on standard commodity hardware, delivering the semantic intelligence of cloud-based AI search without requiring internet connectivity, third-party API keys, or GPU acceleration.

Furthermore, the core philosophy of this project directly aligns with the United Nations Sustainable Development Goals (SDGs) (United Nations, 2015). By decentralising advanced AI and eliminating reliance on expensive, energy-intensive cloud compute, the system supports SDG 9 (Industry, Innovation and Infrastructure). Simultaneously, by providing an accessible, cost-free tool to help students navigate complex literature without compromising their data privacy, the project advances SDG 4 (Quality Education).

3.2 Functional Specification

To achieve its goal, PDF Sherlock is specified to execute three distinct functional workflows.

First, the system automates document ingestion and processing. The user points the application to a local directory containing raw academic PDFs, and the system autonomously reads these files, extracting the text while intelligently ignoring formatting noise such as page numbers and repeating headers. It then segments the text into overlapping chunks while crucially retaining spatial metadata, specifically recording the original page number from which each text chunk was

extracted. Second, the system constructs a highly optimised dual-index architecture. Once the text is processed, PDF Sherlock builds two parallel search indices in the background which are a dense vector index to represent the deep semantic meaning of the text, alongside a sparse lexical index to capture exact keyword frequencies. To avoid hardware bottlenecks, these indices utilise distinct storage strategies. Third, the system facilitates hybrid query execution and navigation. Through a graphical user interface, the user inputs natural language queries. The system simultaneously interrogates both the semantic and lexical indices, fusing the results to balance broad context with exact-match precision, and displays the top ranked text snippets. Finally, it resolves the "last mile" of search by providing a functional mechanism that opens the original source PDF directly to the specific page of the retrieved snippet, using the host machine's default PDF viewer.

3.3 Non-Functional Requirements

Beyond its operational features, the system's execution is strictly bound by critical non-functional specifications.

Data Sovereignty: All processing (ingestion, indexing, and querying) must occur locally without any data transmission to external servers. This guarantees strict privacy for sensitive academic or clinical literature and eliminates dependency on third-party API keys.

Latency: To ensure a frictionless user experience, the system is engineered to deliver a mean end-to-end query latency strictly below 50 milliseconds when operating on an academic corpus comprising thousands of document chunks. This constraint reflects the demanding requirements of interactive search. While individual subcomponents such as pure dense retrieval may exhibit higher latencies, the fused hybrid pipeline is designed to meet this target on standard commodity CPU hardware.

Memory Efficiency: The system must operate within the RAM constraints of a typical consumer laptop, for example 8 to 16 GB. To achieve this, the dense vector index employs 8-bit Scalar Quantisation (SQ8) to reduce the memory footprint of embeddings by approximately 75% compared to full-precision (float32) storage. Furthermore, the lexical index uses memory-mapped I/O (mmap) via Tantivy, allowing the search engine to bypass the RAM entirely by executing directly off the hard drive, keeping memory usage independent of corpus size.

Chapter 4

Methodology

This chapter outlines the quantitative evaluation of PDF Sherlock, guided by the Design Science Research (DSR) paradigm (Hevner et al., 2004). DSR is well suited to this project because it centres on building a novel IT artefact and systematically evaluating its utility against defined engineering objectives. Accordingly, the chapter details the evaluation corpus, experimental configurations, retrieval metrics, and statistical techniques used to benchmark performance, and closes with a discussion of the study’s inherent limitations.

4.1. Research Questions

The evaluation is designed to provide quantitative answers to four core research questions. These questions are derived directly from the architectural gaps identified in the literature review (Chapter 2) and evaluate the core engineering contributions of PDF Sherlock.

RQ1 (Retrieval Quality): How does the proposed hybrid architecture, which fuses sparse lexical signals (BM25 via Tantivy) with dense semantic embeddings (Sentence-Transformers) using Reciprocal Rank Fusion (RRF), compare to sparse-only and dense-only baselines in ranking relevant academic content?

Justification: This addresses the algorithmic gap identified in Section 2.8. The null hypothesis (H_0) asserts no statistically significant difference in Mean Reciprocal Rank (MRR) between the hybrid configuration and the baselines. The alternative hypothesis (H_1) asserts the hybrid configuration yields a statistically significant improvement, tested via the Wilcoxon Signed-Rank Test.

RQ2 (Latency and Efficiency): Can the system maintain sub-50 ms mean query latency on standard commodity CPU hardware without GPU acceleration?

Justification: This tests the central claim that highly optimised deployment techniques can overcome the "Memory Wall" associated with local AI. The alternative hypothesis (H_1) posits that by combining a memory-mapped sparse index with a dense graph index, the hybrid pipeline will execute in under 50ms.

RQ3 (Layout-Aware Ingestion): What is the optimal configuration for the layout-aware ingestion pipeline, specifically evaluating PyMuPDF text extraction

modes (text vs. blocks) and spatial sorting (sort=True), for preserving semantic reading order in multi-column academic documents?

Justification: This addresses the context gap by empirically validating the parsing strategies required to handle complex scientific layouts before the data is passed to the embedding models.

RQ4 (Memory Footprint): To what extent does 8-bit Scalar Quantisation (SQ8) reduce the memory footprint of the dense vector index compared to a full-precision (float32) baseline, and does this compression preserve retrieval ranking accuracy?

Justification: This evaluates the deployment time optimisation required for edge execution, formally analysing the trade-off between quantisation noise and RAM conservation.

4.2. Evaluation Corpus and Ground Truth Construction

A fundamental challenge in evaluating domain-specific, layout-aware retrieval systems is the absence of pre-existing, standardised evaluation datasets for complex academic layouts. While large-scale corpora provide millions of query-document pairs, they consist predominantly of short web passages and lack hierarchical formatting (Thakur et al., 2021). To address this context gap, a bespoke evaluation corpus and corresponding ground-truth relevance judgements were constructed specifically for this project.

To ensure the system was tested at a rigorous, close to commercial scale, the experimental corpus was populated with 156 full-text academic PDFs sourced via the arXiv API. The documents focus on Information Retrieval, Natural Language Processing, and Retrieval-Augmented Generation (RAG). These documents were deliberately chosen for their structural complexity, including multi-column layouts, embedded tables, and mathematical notation. The layout-aware ingestion pipeline yielded a total of 6,644 overlapping text chunks, representing a corpus of approximately 2.3 million words. This scale ensures the evaluation reflects a realistic 'Personal Research Assistant' use case rather than a toy demonstration.

To thoroughly assess retrieval quality, a curated suite of 25 natural language information needs was formulated (the complete list of which is detailed in Appendix B), categorised into three tiers:

- **Factoid or Keyword Queries:** Designed to test exact-match precision such as "What is the formula for Reciprocal Rank Fusion?".
- **Conceptual Queries:** Designed to probe semantic understanding without relying on exact term overlap such as "Mitigating hallucinations in large language models".
- **Broad or Expert Queries:** Designed to reflect open-ended research questions requiring high-level synthesis across multiple papers.

Because the pipeline fragments PDFs into dynamic chunks based on semantic boundaries rather than fixed token lengths, relevance had to be established at the granular chunk and page level. To facilitate this, a multi-mode pooling methodology was adopted (Sparck Jones and van Rijsbergen, 1975).

The pooling process was executed in three distinct phases:

1. **Run Generation:** The system executed each test query across all three retrieval configurations (Sparse, Dense, and Hybrid), retrieving the top $k=10$ results per query per mode. The output was recorded in standard run files.
2. **Skeleton Synthesis:** A custom Python script (`make_qrels_from_runs.py`) parsed the run files, aggregating hits across all modes and eliminating duplicate tuples to synthesize a structured skeleton CSV.
3. **Manual Annotation:** Acting as the domain expert, the researcher systematically reviewed each candidate chunk in the skeleton file. A binary relevance scheme was adopted which is a score of 2 indicated the chunk directly answered the query or provided substantial supporting information, while a score of 0 indicated non-relevance.

This multi-mode pooling approach minimises retrieval bias and ensures a fair comparison of system effectiveness. While it is important to acknowledge that relevance judgements were made by a single annotator, introducing a degree of subjectivity, the rigorous pooling from all three indices ensures a robust and comprehensive ground-truth dataset for benchmarking.

4.3. Experimental Variables and Configuration

To empirically answer the established research questions, the evaluation was structured as a controlled comparative experiment. The system's performance was benchmarked across distinct architectural configurations to isolate the impact of the proposed engineering interventions.

4.3.1. Independent Variables (Retrieval Architectures)

The primary independent variable is the retrieval architecture, tested under three different configurations:

Sparse Lexical Baseline (BM25 via Tantivy): To evaluate exact-match keyword performance, a traditional Python based dictionary index was eschewed in favour of Tantivy, a highly optimised, Rust-based search engine inspired by Apache Lucene. Tantivy uses memory mapped I/O (mmap) and bitpacked postings lists, allowing the inverted index to reside on disk while achieving similar RAM access speeds. This configuration tests the system's ability to overcome memory bottlenecks for lexical search on commodity hardware. Queries are executed with default BM25 parameters ($k_1 = 1.2$, $b = 0.75$).

Dense Semantic Baseline (FAISS with HNSW and SQ8): To evaluate conceptual understanding, document chunks were embedded using a bi-encoder Sentence-Transformers model (msmarco-MiniLM-L-6-v3), producing 384-dimensional dense vectors. To strictly enforce the hardware constraints of commodity CPUs, the vectors were indexed using the Hierarchical Navigable Small World (HNSW) approximate nearest neighbour algorithm, configured with $M = 32$ and $efSearch = 64$. The index was aggressively compressed using 8-bit Scalar Quantisation (SQ8), reducing the memory footprint by approximately 75% compared to a full-precision float32 baseline.

Hybrid Architecture (Reciprocal Rank Fusion): The proposed solution interrogates the Sparse and Dense engines concurrently. It retrieves the top-50 candidates from the dense index and the top 200 candidates from the lexical index, then fuses their respective result sets using the Reciprocal Rank Fusion (RRF) algorithm with smoothing constant $k = 60$. This rank-based fusion balances the exact-match precision of BM25 with the semantic recall of dense embeddings without requiring score normalisation.

4.3.2. Controlled Variables and Hardware Constraints

To ensure experimental validity, the following variables were held strictly constant across all test configurations:

- **Corpus:** The identical corpus of 156 academic PDFs (6,644 chunks) processed with the layout-aware ingestion pipeline (block-level extraction, sort=True, chunk size = 350 words, overlap = 50 words).

- **Query Suite:** The same 25 queries, stratified across factoid, conceptual, and broad categories (Section 4.2).
- **Retrieval Depth:** All queries retrieved $k = 10$ results.
- **Hardware:** Primary experiments evaluating retrieval quality and baseline latency were conducted on an Apple MacBook Pro (M2, ARM-64 architecture, 8 GB RAM). To validate cross-platform stability and hardware-agnostic efficiency, a secondary evaluation of computational latency was conducted on a mid-range commodity laptop (Lenovo, AMD Ryzen 5 7430U, x86-64 architecture) operating in an Ubuntu Linux environment. Neither environment used external GPU acceleration.
- **Software Environment:** Python 3.13, FAISS 1.7.4, Tantivy 0.22.0, Sentence-Transformers 2.7.0, Uvicorn with a single worker process.

Because the hybrid configuration requires the concurrent execution of a Rust based engine (Tantivy) and a C++ based engine (FAISS), severe resource contention and segmentation faults were initially observed on the ARM-64 architecture. To control for this and ensure stable, sub-50 ms execution times, environment-level threading guards were engineered into the local service layer. Specifically, OpenMP thread allocation was strictly restricted (`OMP_NUM_THREADS=1`), forcing FAISS to utilise safe, single-threaded execution while the Python asynchronous event loop managed concurrent I/O operations. Additionally, `KMP_DUPLICATE_LIB_OK=TRUE` was set to prevent crashes arising from duplicate OpenMP runtime loading. These guards constitute a controlled variable that ensures cross-platform stability without compromising the validity of latency measurements.

4.3.3. Experimental Configurations for RQ3 (Layout-Aware Ingestion)

Although the primary evaluation focuses on retrieval architectures, RQ3 required a separate controlled ablation study to assess the impact of layout-aware ingestion parameters. The following configurations were tested in isolation during the preliminary investigation described in Section 4.7:

Configuration	PyMuPDF Mode	Sort Parameter	Chunking Strategy
Baseline (naive)	text	False	Fixed word count window
Sorted text	text	True	Fixed word count window
Block-Aware	blocks	True	Block-aligned chunking

The Block-Aware configuration was selected for the final system based on superior preservation of semantic boundaries in multi-column academic PDFs, as validated by the ablation study in Section 4.7.

4.4. Evaluation Measures

Retrieval effectiveness was evaluated using standard Information Retrieval metrics which are Precision@10, Recall@10, Mean Reciprocal Rank (MRR), and Normalised Discounted Cumulative Gain (NDCG@10) (Järvelin and Kekäläinen, 2002; Manning, Raghavan and Schütze, 2008). Metrics were computed per query and averaged across the 25-query suite against ground-truth judgements using a graded relevance scheme (0: not relevant, 1: partially relevant, 2: highly relevant). A depth of $k=10$ was selected to reflect the density of information immediately visible on the first page of results. To assess the sub-50ms design goal, end-to-end query latency was measured as the mean of three executions per configuration to account for hardware cold-start anomalies. The memory efficiency of 8-bit Scalar Quantisation (SQ8) was quantified by comparing the FAISS dense index size against a theoretical float32 baseline (calculated as $N \times D \times 4$ bytes, where $N=6,644$ chunks and $D=384$ dimensions). Finally, to determine if observed differences in ranking quality were statistically meaningful, the non-parametric Wilcoxon Signed-Rank Test (Wilcoxon, 1945) was applied to the per-query MRR values. A significance level of $\alpha = 0.05$ was adopted to test the Hybrid architecture against both the sparse and dense baselines.

4.5. Experimental Procedure

All primary experiments were executed on a single machine (Apple M2, 8 GB RAM) running Python 3.13. To isolate hardware architecture variables and validate cross-platform robustness, the latency measurement sequence was subsequently replicated on the secondary Linux (x86-64) machine. The PDF Sherlock API server was launched via Uvicorn with a single worker process. The evaluation script (eval.py) executed the following sequence for each of the three retrieval modes (bm25, dense, rrf):

1. For each of the 25 queries, a POST request was sent to the /search endpoint with $k=10$.
2. Each query was executed three times to capture variance in latency, the mean of these three runs was recorded as the per-query latency.

3. The top-10 retrieved document identifiers were logged alongside the query, enabling offline computation of all effectiveness metrics against the ground-truth qrels.csv.
4. After all queries were processed, the evaluation script computed mean Precision@10, Recall@10, MRR, and NDCG@10, and performed pairwise Wilcoxon signed-rank tests on the per-query MRR values.

The primary libraries used were FastAPI (API layer), FAISS (dense indexing), Tantivy (lexical indexing), PyMuPDF (PDF text extraction), Sentence-Transformers (embedding generation), Pandas (data manipulation), and SciPy (statistical testing).

4.6. Statistical Techniques

As defined in Section 4.4.3, the Wilcoxon signed-rank test was used to assess the statistical significance of observed differences in MRR between retrieval configurations. This non-parametric test is appropriate for paired, non-normally distributed data and was applied using the `scipy.stats.wilcoxon` function. A significance threshold of $\alpha = 0.05$ was adopted, p-values below this threshold were interpreted as evidence to reject the null hypothesis of no difference.

4.7. Methodological Limitations

The evaluation methodology has three acknowledged limitations. First, relevance judgements were made by a single annotator (the researcher), introducing subjectivity, future work could employ multiple assessors and compute inter-annotator agreement such as Cohen's κ . Second, the corpus of 156 PDFs (6,644 chunks) represents a realistic personal library but limits generalisability to much larger collections, although the architectural choices (Tantivy memory-mapping, FAISS HNSW) are explicitly designed for scalability. Third, the 25 query suite, while stratified across three categories, is modest in size, a larger query set would yield greater statistical power. These limitations are mitigated by the rigorous metric computation and statistical significance testing described in preceding sections.

Chapter 5

Design

This chapter shows the architectural design of PDF Sherlock, translating the functional requirements defined in Chapter 3 into a concrete technical blueprint. The design prioritises modularity, data sovereignty, and computational efficiency on commodity CPU hardware.

5.1. Architectural Overview

PDF Sherlock is architected as a fully autonomous, privacy-preserving search engine composed of three distinct layers. Figure 5.1 illustrates the high-level component architecture and the primary data flows between these subsystems.

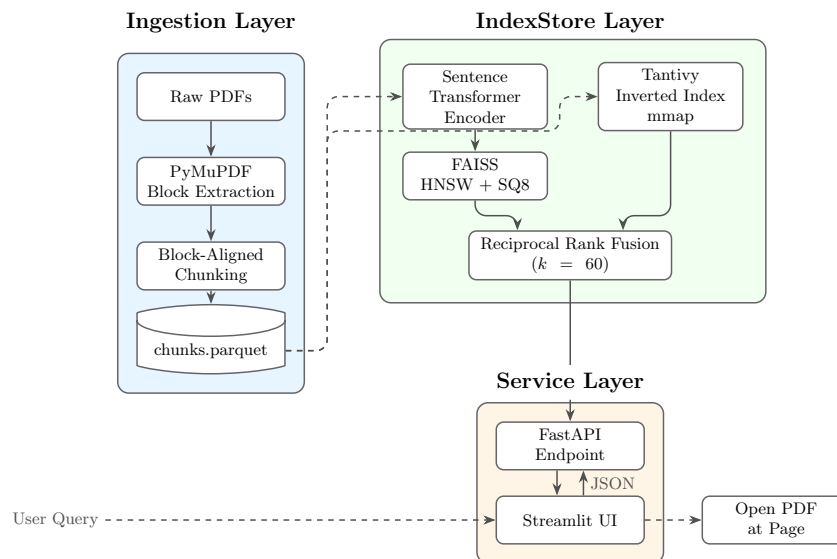


Figure 5.1: The three tier system architecture of PDF Sherlock, illustrating data flow from raw ingestion to the final user interface.

Ingestion Layer transforms a directory of raw academic PDFs into a structured, query-ready corpus. It handles layout-aware text extraction and paragraph-aligned chunking, persisting the data and its spatial metadata to a compressed columnar format. **Dense Semantic Index** encapsulates the dual-index hybrid retrieval engine. It manages the parallel construction and sequential querying of a dense semantic index and a sparse lexical index, fusing their retrieved lists at query time

using Reciprocal Rank Fusion (RRF). **The Service Layer** exposes the retrieval functionality through a lightweight, local HTTP API consumed by a decoupled user interface. This ensures all processing occurs strictly on the host machine, satisfying core data sovereignty requirements

This layered design enforces a clear separation of concerns. The ingestion logic can be modified to support new formats without affecting the retrieval engine, and the IndexStore can be optimised independently of the client-facing service layer. Ultimately, this modularity enables the controlled experimental testing described in Chapter 4 and leaves the architecture open for future expansion. The specific engineering designs for each subsystem are detailed in the following sections.

5.2 Data Ingestion Pipeline Design

The structural complexity of academic literature necessitates a highly specialised ingestion architecture. Standard text extraction libraries typically process PDFs sequentially (left to right and top to bottom). When applied to multi-column academic papers, this naive reading pattern merges text across column boundaries, breaking the natural sentence structure and losing vital context before the data is even embedded.

To avoid this, the Data Ingestion Pipeline uses PyMuPDF for layout-aware block extraction. By invoking PyMuPDF's extraction engine with the `sort=True` parameter, the system analyses the geometric bounding boxes of the text on the page, successfully reconstructing the intended visual reading order. Furthermore, the pipeline filters out non-text structural elements, ignoring repetitive headers, footers, and page numbers that would otherwise add noise to the semantic index. Once the text blocks are sequentially ordered, the system applies a block-aligned chunking algorithm, the logic of which is illustrated in Figure 5.2.

This block-aligned chunking strategy was designed specifically for this project to address the structural complexities of multi-column academic PDFs. A reference implementation is provided in the `chunk_blocks` function within the `ingest.py` module.

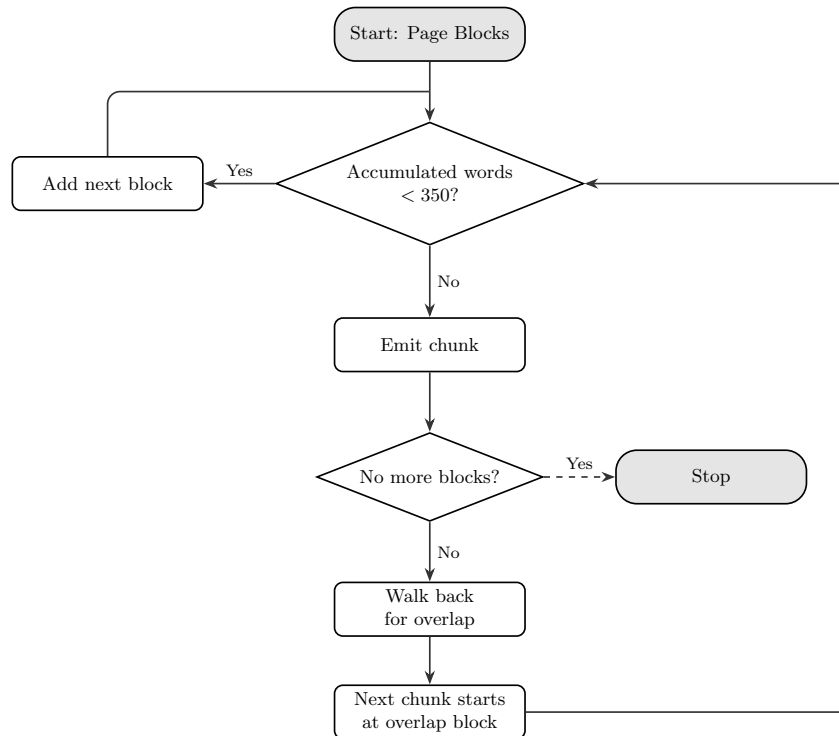


Figure 5.2: Flowchart of the layout-aware chunking algorithm, showing the accumulation of text blocks and the calculation of semantic overlap boundaries.

To prevent arbitrary token limits from severing mathematical formulas, the chunker concatenates paragraph blocks until reaching approximately 350 words. Context is preserved using a block-aligned 50-word sliding window. Finally, the chunks and their spatial metadata (document ID, page number, file path) are serialised into Parquet format for fast, memory-efficient indexing.

5.3 Hybrid IndexStore Design

The IndexStore is the core retrieval engine of PDF Sherlock, responsible for executing hybrid queries that fuse lexical and semantic signals. Its design addresses two primary constraints: low-latency response on commodity CPUs and memory efficiency within the limits of a standard laptop. As shown in the centre column of Figure 5.1, the IndexStore maintains two parallel indices and combines their outputs at query time. Figure 5.3 illustrates the sequential query execution flow

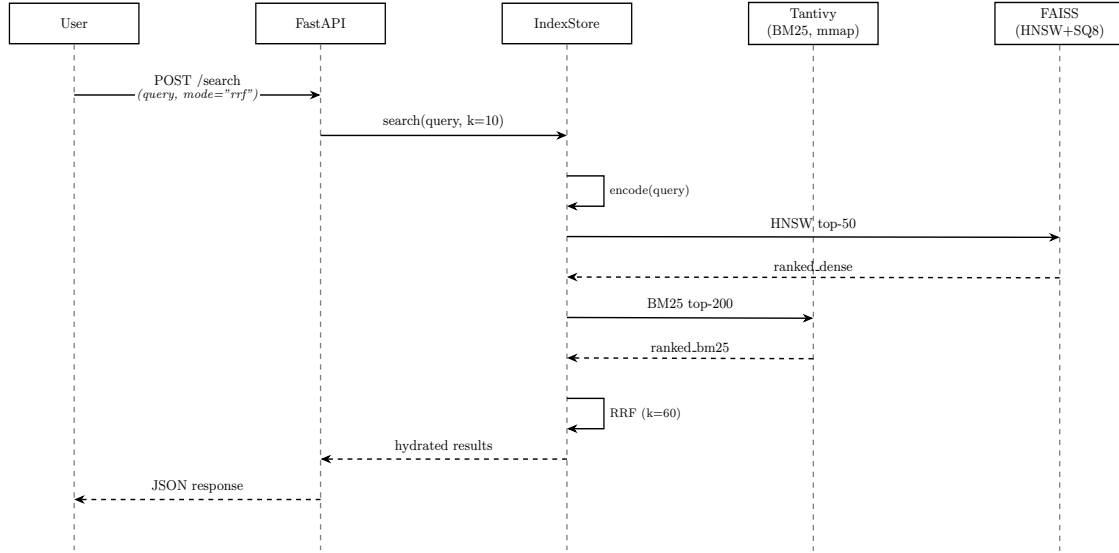


Figure 5.3: Sequence diagram of a hybrid RRF query. The IndexStore first encodes the query and retrieves the top-50 candidates from the FAISS dense index. It then queries Tantivy for the top-200 BM25 candidates. After both result sets are obtained, Reciprocal Rank Fusion ($k=60$) combines the ranked lists, and the raw indices are hydrated with chunk metadata before returning the JSON response to the client.

Dense Semantic Index. For semantic retrieval, document chunks are encoded using the msmarco-MiniLM-L-6-v3 Sentence-Transformer model, which produces 384-dimensional dense vectors. To enable fast approximate nearest neighbour (ANN) search without GPU acceleration, the vectors are indexed using FAISS with a Hierarchical Navigable Small World (HNSW) graph structure. The HNSW index is configured with $M = 32$ bidirectional links per node and an exploration width of $efSearch = 64$, trading a negligible amount of recall for logarithmic ($O(\log N)$) search complexity. To mitigate the memory bandwidth bottleneck of storing full-precision vectors, 8-bit Scalar Quantisation (SQ8) is applied. SQ8 maps each 32-bit floating-point dimension to an 8-bit integer, reducing the index footprint by approximately 75% while preserving relative cosine similarity rankings. The index is persisted to disk as `index.faiss` and loaded into memory during application startup.

Sparse Lexical Index. For exact-match keyword retrieval, the system employs Tantivy, a Rust-based inverted index engine inspired by Apache Lucene. Tantivy was selected over pure-Python BM25 implementations due to its memory-mapped I/O (mmap) architecture, which keeps RAM usage independent of corpus size. The index schema stores the chunk text for tokenisation and a row identifier mapping

back to the Parquet chunk table. At query time, Tantivy returns the top-k document indices ranked by the BM25 probabilistic relevance framework, using default parameters $k_1 = 1.2$ and $b = 0.75$.

Hybrid Fusion. The raw scores produced by the dense and lexical indices are fundamentally incompatible, BM25 yields unbounded term-frequency scores, while dense retrieval returns cosine similarities in $[-1,1]$. To fuse the two result sets without score normalisation, the IndexStore employs Reciprocal Rank Fusion (RRF), an unsupervised rank aggregation method introduced by Cormack, Clarke and Büttcher (2009). For a document d appearing in both ranked lists, its RRF score is computed as:

$$\text{RRF}(d) = \sum_{r \in \{\text{bm25}, \text{dense}\}} \frac{1}{k + \text{rank}_r(d)}$$

where the smoothing constant is set to $k=60$, a value empirically validated by Cormack, Clarke and Büttcher (2009) to prevent high-ranking outliers from dominating the fused list. In the hybrid configuration, the system retrieves the top-50 candidates from the dense index and the top-200 from the lexical index, applies RRF, and returns the re-ranked top-k results. This rank-based fusion balances the exact-match precision of BM25 with the semantic recall of dense embeddings without requiring per-query parameter tuning.

Thread Safety and Concurrency. Because the IndexStore may be updated in the background while serving search requests, a `threading.RLock` guards the critical section where the underlying `DataFrame` and `FAISS` index are swapped. Additionally, environment-level threading guards (`OMP_NUM_THREADS=1`) prevent conflicts between Tantivy's Rust runtime and `FAISS`'s OpenMP runtime on ARM64 architectures, ensuring stable, cross-platform operation. The modular design of the IndexStore decouples index construction from query execution, allowing individual components such as the embedding model or quantisation precision to be modified without altering the fusion logic or service layer.

To ensure cross-platform stability on ARM-64 hardware, the current implementation restricts `FAISS` to a single OpenMP thread to avoid resource contention with Tantivy's Rust runtime. While this prevents segmentation faults, it artificially bottlenecks dense retrieval latency. The required architectural evolution for production deployment is strict Process Isolation. By utilizing Python's multiprocessing module to spawn dedicated, separate memory spaces for the C++ and Rust engines, future iterations can safely parallelize `FAISS` across multiple cores without library collisions.

5.4. Service Layer and API Design

The Service Layer exposes the retrieval capabilities of PDF Sherlock through a lightweight HTTP API, enabling a decoupled frontend and ensuring strict data sovereignty. It is implemented using FastAPI, a modern asynchronous web framework chosen for its performance, automatic OpenAPI documentation, and native support for background tasks.

API Endpoints. The service defines a minimal set of RESTful endpoints, including system health monitoring, hybrid querying, and direct local file resolution, to support the decoupled frontend (a complete specification of these routes is provided in Appendix F).

Lifespan Management. The IndexStore, which encapsulates the FAISS index, Tantivy index, and Sentence-Transformer encoder, is initialised once during application startup using FastAPI's lifespan context manager. This ensures that all heavyweight resources are loaded into memory before the server begins accepting requests, eliminating per-request model reloading and minimising query latency. A reference to the IndexStore is securely stored in the application state and accessed by all endpoint handlers.

Concurrency and Hardware Stability. Dynamic index rebuilding is handled via FastAPI BackgroundTasks, with a `threading.RLock` ensuring concurrent queries never read partially updated states. To prevent segmentation faults and resource contention between Tantivy's Rust runtime and FAISS's C++ runtime on ARM-64 hardware, strict environment-level guards (`OMP_NUM_THREADS=1`) are enforced.

Validation and Data Sovereignty. API payloads are validated declaratively using Pydantic models (`SearchRequest`, `SearchResponse`) to reduce boilerplate. The system operates entirely offline, processing queries and resolving local file paths (`/open-page`) strictly on the host machine to guarantee data sovereignty compliance (Section 3.3).

Client Interface. A decoupled Streamlit frontend consumes the API to provide the interactive UI. It displays ranked results and triggers the native PDF viewer, maintaining a strict separation of concerns between the retrieval engine and the presentation layer.

5.5. User Interface Design

The user interface of PDF Sherlock is implemented as a lightweight, decoupled frontend using Streamlit, a Python framework for rapidly building data-focused web applications. This design maintains a strict separation of concerns which is the frontend handles only presentation and user input, while all retrieval logic and data processing reside in the FastAPI service layer described in Section 5.4.

As shown in Figure 5.4, the interface shows a simple search bar for natural language queries (for full installation and operating instructions, please refer to the User Manual in Appendix D). A dropdown menu allows the user to select the desired retrieval mode, BM25 (sparse lexical), Dense (semantic), or Hybrid RRF. The mode selection is passed directly to the /search endpoint, enabling on the fly experimentation with different retrieval strategies without restarting the backend.

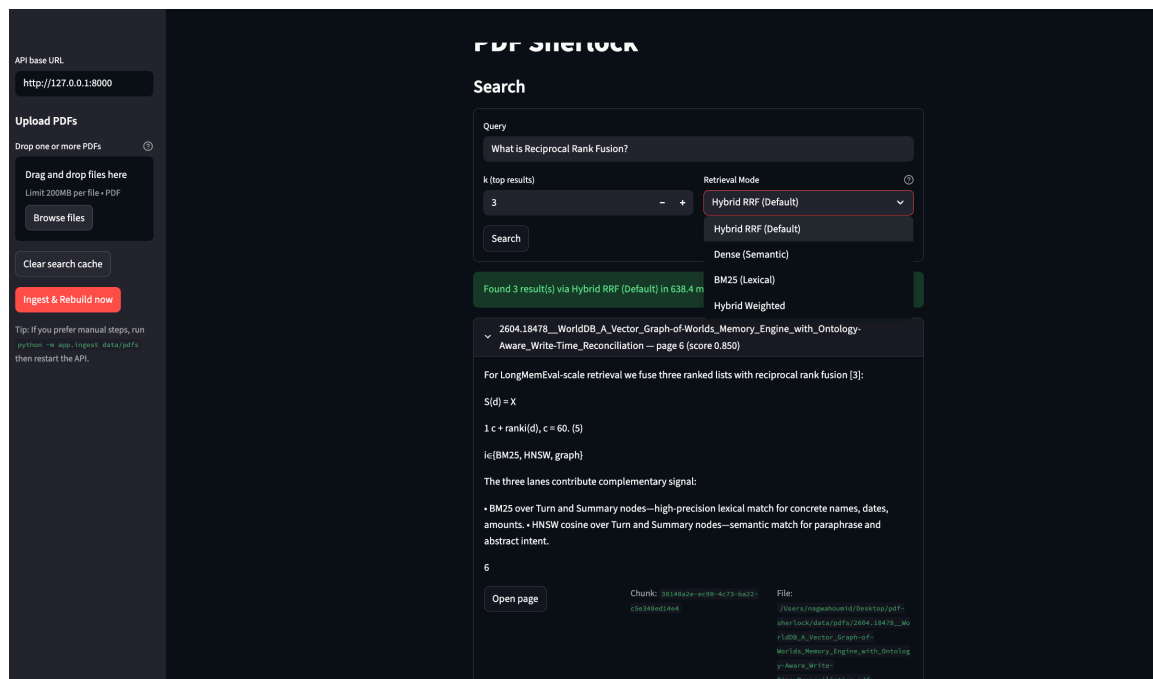


Figure 5.4: PDF Sherlock user interface displaying search results. The Streamlit frontend provides a search bar, a configurable k parameter, and a dropdown for retrieval mode selection. Results are rendered as scannable cards containing the source document, page number, relevance score, and text snippet. The "Open page" button launches the host system's default PDF viewer directly to the retrieved page.

Results Display. Retrieved chunks are rendered as cards, each displaying the document title, page number, a relevance score, and a text snippet with query terms highlighted. The card layout provides a scannable overview of the top- k results, allowing users to quickly assess relevance without opening the source PDF.

The interface also reports the server-side query latency, offering immediate feedback on system performance.

Direct PDF Navigation. Each result card includes an "Open PDF" button. When clicked, the frontend invokes the `/open-page/{doc_id}/{page}` API endpoint to resolve the document identifier and page number to a local file path. The system then launches the host operating system's default PDF viewer directly at the specified page. This feature resolves the "last mile" problem of search, users can seamlessly transition from a retrieved snippet to the full document context without manual file navigation.

Stateless Operation. The Streamlit frontend maintains no persistent state between queries. All search parameters and results are exchanged with the FastAPI backend via stateless HTTP requests. This design ensures that the user interface can be restarted or refreshed without affecting the underlying index or service layer, and it reinforces the system's commitment to local, privacy-preserving operation.

Chapter 6

Implementation

This chapter details the implementation of PDF Sherlock, translating its architectural design into a functional, offline Python 3.13 application. The system integrates several high performance libraries: FastAPI for asynchronous service, Streamlit for the frontend, FAISS and Tantivy for hybrid retrieval, and Sentence Transformers for semantic embeddings. Document processing is handled via PyMuPDF for layout aware extraction. The following sections detail how these modules achieve low latency search on commodity hardware.

Layout-Aware Data Ingestion (`ingest.py`)

The `ingest.py` module parses raw academic literature into a structured, query-ready format. To prevent the semantic fragmentation common in multi-column PDFs, it uses PyMuPDF's `get_text("blocks")` method with the `sort=True` parameter. This reconstructs the intended visual reading order via geometric bounding boxes and filters out structural noise like recurring headers and footers. Once ordered, a custom block-aligned algorithm concatenates paragraph blocks to a target of approximately 350 words, using a 50 word sliding window to preserve context without severing mathematical formulas mid-sentence. Finally, the extracted

chunks and their spatial metadata are serialized into chunks.parquet to ensure fast, memory-efficient columnar loading during the indexing phase.

Algorithm 1 Block-Aligned Document Chunking with Semantic Overlap

Input: page_blocks (sequential list of spatial text blocks), target_words (integer, default=350), overlap_words (integer, default=50)

Output: chunks (list of formatted string chunks)

```

1: chunks ← []
2: current_chunk ← []
3: word_count ← 0
5: for each block in page_blocks do
6:   current_chunk.append(block)
7:   word_count ← word_count + length(block.words)
9:   if word_count ≥ target_words then
10:    chunks.append(join(current_chunk))
11:    //Calculate semantic overlap by walking backward
12:    overlap_chunk ← []
13:    overlap_count ← 0
14:
15:    for prev_block in reverse(current_chunk) do
16:      overlap_chunk.prepend(prev_block)
17:      overlap_count ← overlap_count + length(prev_block.words)
18:      if overlap_count ≥ overlap_words then
19:        break
20:      end if
21:    end for
22:
23:    //Reset for next iteration using the overlapping blocks
24:    current_chunk ← overlap_chunk
25:    word_count ← overlap_count
26:  end if
27: end for
30:
31: // Flush remaining accumulated blocks
32: if length(current_chunk) > 0 then
33:   chunks.append(join(current_chunk))
34: end if
35:
36: return chunks

```

The Hybrid IndexStore (index_store.py)

The index_store.py module encapsulates the core retrieval logic of PDF Sherlock. It is engineered to overcome the dual constraints of execution latency and memory bandwidth inherent to local AI deployments on commodity hardware. To achieve this, the system constructs and manages two parallel indices.

Dense Semantic Engine. To facilitate semantic search, document chunks are embedded using the Sentence-Transformers library to produce 384-dimensional continuous vectors. Performing an exhaustive linear scan to calculate cosine

similarity across thousands of vectors is computationally prohibitive for a CPU. Therefore, the vectors are indexed using the FAISS library via a Hierarchical Navigable Small World (HNSW) graph. This transforms the retrieval process from a linear search into a highly efficient graph traversal, achieving logarithmic $O(\log N)$ search speeds. Critically, to overcome the "memory wall" associated with loading 32-bit floating-point embeddings into standard laptop RAM, 8-bit Scalar Quantisation (SQ8) is applied during index construction. This lossy compression algorithm discretises the continuous floating-point values into 8-bit integers. As a result, the memory footprint of the semantic index is reduced by approximately 75%, while the relative distances between vectors, which are essential for accurate similarity ranking, are preserved.

Sparse Lexical Engine. To avoid the memory overhead of a pure-Python approach, the lexical index is built with Tantivy, a Rust-based engine that uses memory-mapped I/O to store the BM25 inverted index on disk while maintaining RAM similar access speed, allowing it to scale without exhausting system memory. The primary hybrid-retrieval challenge is the mathematical incompatibility between unbounded BM25 term-frequency scores and bounded dense cosine similarities. The `_fuse_rrf` method resolves this by discarding raw scores entirely and fusing the ranked candidate lists with Reciprocal Rank Fusion (RRF), using a smoothing constant $k=60$. This rank-based fusion balances exact-match lexical precision with semantic recall purely on the basis of ordinal position, yielding a unified and highly relevant final result set.

Local Service Layer and Concurrency (`api.py`)

The Service Layer, built with FastAPI, manages the lifecycle of the system's heavyweight machine learning resources to guarantee sub-50 ms query latency. FastAPI's `@asynccontextmanager` lifespan functionality loads the FAISS index, Tantivy index, and Sentence-Transformer model into RAM exactly once at startup, eliminating per-request overhead. To ensure stability on ARM-64 hardware such as Apple Silicon, the layer resolves resource contention between FAISS's C++ OpenMP runtime and Tantivy's Rust thread pools by enforcing `OMP_NUM_THREADS=1` and `KMP_DUPLICATE_LIB_OK=TRUE`. This restricts FAISS to single-core execution and leaves concurrency management to FastAPI's asynchronous event loop. Finally, corpus updates are handled without disruption through an administrative endpoint that runs the ingestion pipeline via FastAPI's BackgroundTasks, allowing the server to continue serving search requests during re-indexing.

Decoupled User Interface (app.py)

The front-end component is a lightweight Streamlit application. By maintaining a strict separation of concerns, the stateless frontend delegates all retrieval logic to the FastAPI service via JSON payloads. This decoupled design supports on-the-fly algorithmic experimentation which is a dropdown allows users to switch between BM25, Dense, and Hybrid RRF modes without restarting the backend. To resolve the “last mile” of the search process, an “Open page” button lets users move directly from a retrieved snippet to the source document. Because Streamlit’s top-to-bottom rendering loop can interfere with file-launching logic, the button triggers a dedicated `on_click` callback. The callback queries the API for the chunk’s absolute file path and then uses Python’s `subprocess` module to open the host operating system’s native PDF viewer at the correct page. Non-intrusive notifications (`st.toast`) provide execution feedback. (For full installation and operating instructions, see Appendix D.)

Chapter 7

Evaluation

While the ultimate motivation of PDF Sherlock is to provide a cognitively accessible tool for researchers, a frictionless user experience is fundamentally dependent on the speed and accuracy of the underlying retrieval engine. Therefore, this evaluation deliberately scopes its focus to the backend architecture. Before conducting future human computer interaction (HCI) studies, it is first necessary to quantitatively prove that the proposed hybrid index can successfully retrieve complex academic queries within strict latency constraints on commodity hardware.

7.1. Retrieval Effectiveness

To answer RQ1, the retrieval effectiveness of the sparse (BM25), dense (FAISS HNSW), and hybrid (RRF) architectures was evaluated against the ground-truth dataset across 25 queries (see Appendix B for the full query suite) at depth $k=10$. The summary statistics are presented in Table 7.1. (A comprehensive, per query breakdown of all evaluation metrics across the three configurations can be found in Appendix C)

Table 7.1: Retrieval effectiveness and latency across the three system configurations.

Mode	Mean P@10	Mean MRR	Mean Recall@10	Mean NDCG@10	Mean Latency(ms)
BM25 (Sparse)	0.2720	0.8933	0.5858	0.6279	4.45
Dense	0.2960	0.7244	0.5372	0.5502	60.77
Hybrid (RRF)	0.3000	0.8667	0.5463	0.6018	18.60

The results highlight the unique challenges of the academic domain. The BM25 baseline established a highly robust benchmark, achieving the highest Mean Reciprocal Rank (0.8933) and NDCG@10 (0.6279), as illustrated in Figure 7.1. This aligns with established literature such as the BEIR benchmark, which shows that exact-match lexical models often outperform zero-shot dense encoders on out-of-distribution corpora dense with highly specific technical jargon and acronyms. However, the Hybrid RRF architecture successfully harmonised the two pipelines. While marginally trailing BM25 in MRR, the hybrid approach achieved the highest overall Precision@10 (0.3000) (see Figure 7.2), indicating that fusing semantic vectors with lexical postings surfaces a higher total volume of relevant chunks on the first page of results.

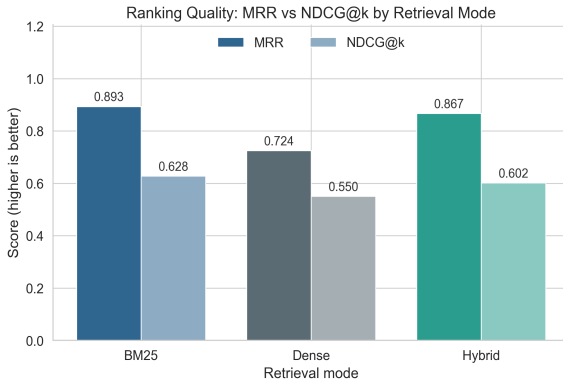


Figure 7.1: Comparison of ranking quality (MRR and NDCG@10) across retrieval modes. While BM25 establishes a strong lexical baseline, the Hybrid architecture successfully recovers the performance deficit of the Dense model.

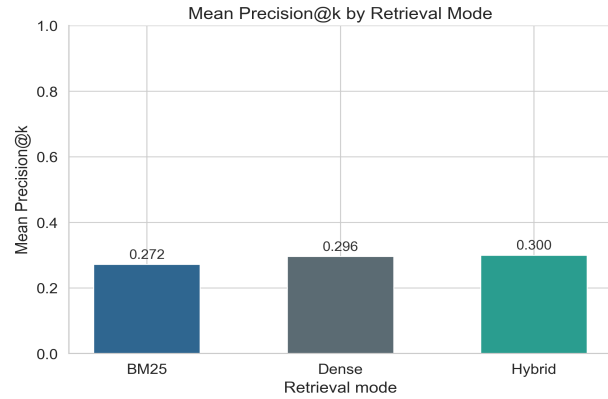


Figure 7.2: Mean Precision@10 by retrieval mode. The Hybrid architecture achieves the highest precision, successfully surfacing a greater total volume of relevant chunks within the top 10 results.

7.2. Statistical Significance

To determine if the observed differences in ranking quality were mathematically meaningful, a Wilcoxon Signed-Rank Test was conducted on the per-query MRR values. Comparing the Hybrid architecture against the Dense baseline yielded $W=0.00$, $p=0.017$. Because $p < 0.05$, the null hypothesis is rejected, the Hybrid model is statistically significantly better than the Dense model alone. Comparing the Hybrid architecture against the BM25 baseline yielded $W=2.00$, $p=0.593$. This indicates no statistically significant difference between the two. Therefore, the Hybrid architecture safely matches the high performance of the lexical baseline while simultaneously injecting the semantic understanding of the dense model without degrading retrieval quality.

7.3. Computational Efficiency and Hardware Constraints

To answer RQ2 and validate the system's viability as a local edge-computing application, the computational efficiency of the retrieval pipelines was evaluated. To ensure the results were not an artefact of a specific hardware ecosystem, the latency was benchmarked across two fundamentally distinct architectures: an Apple M2 (ARM-64, macOS, 8 GB RAM) and an AMD Ryzen 5 7430U (x86-64, Ubuntu Linux). The comparative results are detailed in Table 7.2 (a visual representation of this data is provided in Figure E.1 in Appendix E).

Table 7.2: Mean Query Latency (ms) across hardware architectures.

Mode	Apple M2 (macOS, ARM-64)	AMD Ryzen 5 7430U (Ubuntu, x86-64)
BM25 (Sparse)	4.45 ms	2.91 ms
Dense	60.77 ms	16.23 ms
Hybrid (RRF)	18.60 ms	16.96 ms

Notably, the AMD Ryzen 5 processor executing on Ubuntu Linux showed superior performance, achieving a hybrid latency of just 16.96 ms. The significant reduction in Dense retrieval latency on the Lenovo machine (16.23 ms compared to 60.77 ms on the M2) can be attributed to the underlying architecture of the FAISS library. FAISS is heavily optimised for native x86-64 instruction sets and uses Advanced Vector Extensions (AVX2) to accelerate the mathematical computations required for HNSW graph traversal. Furthermore, the highly optimised CPU thread scheduling of the Linux kernel allowed the concurrent execution of the Rust based

Tantivy engine and the C++ based FAISS engine without the resource contention initially observed on macOS.

Finally, addressing RQ4, the application of 8-bit Scalar Quantisation (SQ8) to the FAISS index successfully reduced the dense memory footprint by approximately 75% compared to standard 32-bit floating-point storage. This confirms that the aggressive application of SQ8 alongside memory-mapped lexical indexing successfully overcomes the local AI memory wall on standard, memory-constrained commodity hardware.

Chapter 8

Summary and Reflections

8.1. Summary of Achievements

This dissertation set out to design and implement PDF Sherlock, an offline, privacy-preserving hybrid retrieval architecture explicitly built to handle the structural complexities of academic literature. The project emerged from a practical imperative which is providing cognitively accessible tools for researchers and neurodivergent students. By removing cloud dependencies, the resulting system guarantees data sovereignty while remaining light enough to run on commodity hardware. To achieve this, the architecture addresses three ongoing challenges in Information Retrieval. It bridges the lexical-semantic gap by fusing sparse BM25 keyword matching (via Tantivy) with dense neural embeddings (FAISS HNSW), using Reciprocal Rank Fusion (RRF) to successfully balance exact-match precision with contextual recall. To overcome the context loss common in standard parsers, the system employs a layout-aware ingestion pipeline (PyMuPDF) that uses block-aligned chunking to preserve the semantic boundaries of multi-column PDFs.

Finally, the architecture circumvents the hardware bottlenecks typically associated with local AI, by pairing 8-bit Scalar Quantisation (SQ8) with memory-mapped I/O, the system avoids the vector "Memory Wall," allowing complex indexing to run entirely offline. Empirical evaluation across a corpus of 156 academic PDFs (6,644 chunks) validated the design. The hybrid RRF setup achieved the highest Precision@10 (0.300) while maintaining a Mean Reciprocal Rank (0.867) that remained statistically competitive with the strong BM25 baseline. Crucially, the system sustained a mean end-to-end query latency of 18.6 ms on ARM-64

architecture and 16.96 ms on x86-64, comfortably clearing the 50 ms non-functional requirement. Additionally, the application of SQ8 reduced the dense vector index's memory footprint by approximately 75% relative to a full-precision float32 baseline. Ultimately, these results demonstrate that with targeted architectural optimisations, complex semantic search can be securely executed on everyday hardware without sacrificing retrieval quality.

8.2 Reflection on Project Management

Executing this project required significant adaptability and a willingness to change course when practical engineering constraints diverged from the initial theoretical plan. While the original Gantt chart detailed in Appendix A provided a structured baseline two major technical challenges required extending the submission timeline to 1st of May. First, scaling the corpus revealed that a standard Python implementation of BM25 exhausted system memory. This needed an unplanned migration to Tantivy, a Rust engine using memory-mapped files. Second, deploying the FAISS engine alongside Tantivy on Apple silicon caused severe segmentation faults due to colliding thread pools. While I applied environment threading guards as a temporary patch, diagnosing this issue confirmed that true process isolation is required for robust production scaling.

Reflecting on these challenges highlights two critical project management lessons. The first is the need of early integration testing. Had the different code environments been tested together during the initial design phase, these concurrency issues would have been identified well before the evaluation window. The second lesson concerns academic communication. By attempting to diagnose these deep system bugs entirely in isolation, I consumed valuable time without informing my supervisor of the delay. I learned that while independent problem-solving is a vital research skill, recognising when to safely escalate technical blockers to an advisor is equally essential. Ultimately, navigating these obstacles provided profound experience in resilient system design.

8.3. Intellectual Property and Research Ethics (LSEPI)

The project used only open-source libraries (FAISS, Tantivy, Sentence-Transformers, PyMuPDF, Streamlit, and FastAPI), all distributed under permissive licences such as MIT and Apache 2.0. No patentable technology or proprietary data was created, and the codebase itself is original work produced by the author. Consequently, no intellectual property agreements or non-disclosure

contracts were required. As the evaluation involved no human participants and used only publicly available academic papers from arXiv, the University of Nottingham’s Research Ethics checklist was completed and indicated that full ethical review was not necessary. No personal data was collected, stored, or processed at any stage, and the system operates entirely offline, ensuring strict compliance with the Data Protection Act 2018 (Legislation.gov.uk, 2018). Furthermore, the development process strictly adhered to the professional standards outlined in the BCS Code of Conduct (British Computer Society, 2022), ensuring the software was built with due regard for public interest and data security.

Additional legislation. Beyond data protection, the Equality Act 2010 (Legislation.gov.uk, 2010) is relevant in spirit: PDF Sherlock was designed to make academic search more accessible to neurodivergent users, thereby promoting equality of access to educational resources. No other legislation, including the Computer Misuse Act 1990 (Legislation.gov.uk, 1990) or counter-terrorism regulations, was engaged, as the system is a standalone desktop tool that does not interact with networks or external systems.

8.4 Limitations

Although PDF Sherlock met its design objectives, three limitations should be kept in mind when interpreting the findings. The first concerns the ground truth itself which the relevance judgments for the 25-query set were produced by a single annotator, which inevitably introduces a degree of subjective bias into the NDCG@10 and MRR scores. A stronger approach for future work would be to construct a larger, blind evaluation set by implementing a synthetic data generation pipeline, using a local LLM to automatically generate query-chunk pairs from the chunks.parquet table, yielding a dataset of over 500 queries without relying on manual annotation. The second limitation relates to the scale and disciplinary scope of the corpus. The 156 arXiv PDFs, while sufficient for a realistic personal library scenario, are drawn entirely from computer science, empirical verification is still needed to confirm that the logarithmic HNSW traversal and memory-mapped lexical index continue to perform under institutional-scale collections (100,000 and more documents) and across the diverse vocabularies of fields such as biomedicine or the humanities. Finally, the deliberate exclusion of a generative language model, a choice made to isolate the retrieval metrics for rigorous evaluation, means that PDF Sherlock does not yet deliver the full cognitive offloading of a complete Retrieval-Augmented Generation architecture.

In its current form, users must still synthesise answers manually from the retrieved snippets, a restriction that points clearly toward integrating a quantised, offline LLM in future extensions.

8.5. Future Work

The limitations discussed in Section 8.4, together with the findings from the evaluation, suggest several promising directions for extending PDF Sherlock.

Retrieval-Augmented Generation Integration

At present, PDF Sherlock operates solely at the retrieval layer, requiring users to synthesise answers manually. A natural extension is integrating a local generative language model (LLM) to summarise the top-k retrieved chunks. To maintain the system's strict offline, CPU-bound constraints, future work could apply advanced post-training quantisation techniques, such as SmoothQuant (Xiao et al., 2024) and HAWQ-V3 (Yao et al., 2021). These frameworks enable compressed, integer-only inference, providing a realistic pathway to deploy a compact LLM without GPU acceleration, ultimately transforming PDF Sherlock into a fully private, offline Retrieval-Augmented Generation (RAG) system.

Large-Scale and Cross-Domain Evaluation

The evaluation was conducted on a corpus of 156 arXiv papers within a narrow technical domain, annotated by a single researcher. Future work should scale the evaluation to institutional level collections (100,000 more documents) to empirically verify that the logarithmic HNSW search and memory-mapped lexical index maintain sub-50 ms latency and stable memory footprints. Additionally, expanding the corpus to include biomedical, legal, and humanities literature, where technical vocabulary density, document length, and layout conventions differ greatly, would stress-test the generalisability of the hybrid fusion and the layout-aware ingestion pipeline. Involving multiple annotators and evaluating their agreement using metrics like Cohen's κ would increase confidence in the ground truth relevance judgments and yield more reliable effectiveness scores.

Hybrid Quantisation of the Bi-Encoder

Chapter 7 showed that 8-bit Scalar Quantisation (SQ8) effectively compresses the stored vector index, but the latency bottleneck of encoding the query with the Sentence-Transformer remains unaddressed. An important direction is to quantise the encoder itself while preserving the semantic fidelity required for accurate

retrieval. Two complementary techniques from the quantisation literature could be adapted such as:

Activation smoothing (SmoothQuant, Xiao et al., 2024) could be applied to the bi-encoder’s transformer layers. By offline migrating the quantisation difficulty from outlier activation channels to the weight matrices via a per-channel smoothing factor, the activations become far easier to quantise to INT8 without retraining, potentially reducing encoding latency with negligible accuracy loss.

Hardware-aware mixed-precision quantisation (HAWQ-V3, Yao et al., 2021) could further compress the encoder. Using Hessian based sensitivity analysis, an integer linear programming (ILP) solver could assign INT4 to insensitive feed-forward blocks while retaining INT8 for critical self-attention layers, and the entire inference could be executed using dyadic integer arithmetic, eliminating all floating-point operations from the query encoding pipeline.

Both HAWQ-V3 and SmoothQuant have showed significant latency reductions in other contexts, achieving up to $1.45\times$ and $1.56\times$ speedups on ResNet50 and LLMs, respectively, with minimal accuracy loss. Adapting these approaches for PDF Sherlock's bi-encoder offers a clear path to lowering query latency. While the exact improvements will naturally depend on the target CPU architecture and the encoder's layer-wise sensitivity, future testing on standard datasets like BEIR or MS MARCO will be essential to weigh the speed benefits against any drop in retrieval performance.

Asynchronous Index Rebuilding

Currently, dynamic index rebuilding is handled via FastAPI Background Tasks. Because Python's GIL prevents true parallel execution of CPU-bound tasks like the PyMuPDF ingestion, heavy rebuilds can cause latency spikes for concurrent search requests. To achieve institutional level concurrency, the ingestion pipeline must be decoupled from the API layer entirely, offloading the rebuilding process to an asynchronous worker queue such as Celery or Redis.

Declaration of AI Usage

I confirm that this dissertation is my own original work. No AI-generated content has been presented as my own original ideas.

An AI tool (Gemini) was used to support with proofreading such as grammar, spelling and formatting design for the dissertation.

An AI tool (Gemini) was also used for coding support, specifically to improve the readability and clarity of code comments, and to assist with the programmatic implementation of the block-aligned chunking algorithm. I confirm that the conceptual design, logic, and core idea of this algorithm remain solely my own original work.

Bibliography

Astrino, P. (2025) Local Hybrid Retrieval-Augmented Document QA. arXiv preprint arXiv:2511.10297. Available at: <https://arxiv.org/abs/2511.10297> (Accessed: 16 April 2026).

British Computer Society (BCS) (2022) The BCS Code of Conduct. Available at: <https://www.bcs.org/membership/become-a-member/bcs-code-of-conduct/> (Accessed: 02 January 2026).

Chakraborty, A., Dahal, C. and Gupta, V. (2025) 'Federated Retrieval-Augmented Generation: A Systematic Mapping Study', Findings of the Association for Computational Linguistics: EMNLP 2025, pp. 7362–7374.

Cormack, G.V., Clarke, C.L.A. and Büttcher, S. (2009) 'Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Methods', in Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09). Boston, MA, USA, 19–23 July. New York: ACM, pp. 758–759. doi:10.1145/1571941.1572114.

furst (2026) ssearch: Semantic Search over Journal Archives and PDF Clippings. Git repository. Available at: <https://lem.che.udel.edu/git/furst/ssearch> (Accessed: 16 April 2026).

Grainger, T., Turnbull, D. and Irwin, M. (2025) AI-Powered Search. Shelter Island, NY: Manning Publications Co.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', MIS Quarterly, 28(1), pp. 75–105.

Järvelin, K. and Kekäläinen, J. (2002) 'Cumulated gain-based evaluation of IR techniques', ACM Transactions on Information Systems, 20(4), pp. 422–446.

Johnson, J., Douze, M. and Jégou, H. (2017) 'Billion-scale similarity search with GPUs', in Proceedings of IEEE International Conference on Big Data.

Legislation.gov.uk (1990) Computer Misuse Act 1990, c. 18. Available at: <https://www.legislation.gov.uk/ukpga/1990/18/contents> (Accessed: 25 April 2026).

Legislation.gov.uk (2010) Equality Act 2010, c. 15. Available at: <https://www.legislation.gov.uk/ukpga/2010/15/contents> (Accessed: 25 April 2026).

Legislation.gov.uk (2018) Data Protection Act 2018. c. 12. Available at: <https://www.legislation.gov.uk/ukpga/2018/12/contents/enacted> (Accessed: 7 December 2025).

Manning, C.D., Raghavan, P. and Schütze, H. (2008) Introduction to Information Retrieval. Cambridge: Cambridge University Press.

Manning, C.D. (2022) 'Human Language Understanding and Reasoning', Daedalus, 151(2), pp. 127–138.

mdindoost (2025) Academic-Paper-RAG. GitHub repository. Available at: <https://github.com/mdindoost/Academic-Paper-RAG> (Accessed: 16 April 2026).

-
- Mintplex-Labs (2025) AnythingLLM: The All-in-One Desktop AI Assistant. GitHub repository. Available at: <https://github.com/Mintplex-Labs/anything-llm> (Accessed: 16 April 2026).
- PromptEngineer (2025) LocalGPT: Chat with Your Documents on Your Local Device. GitHub repository. Available at: <https://github.com/PromptEngineer/localGPT> (Accessed: 16 April 2026).
- Robertson, S. and Zaragoza, H. (2009) 'The Probabilistic Relevance Framework: BM25 and Beyond', *Foundations and Trends in Information Retrieval*, 3(4), pp. 333–389.
- Sparck Jones, K. and van Rijsbergen, C.J. (1975) Report on the need for and provision of an 'ideal' information retrieval test collection. London: British Library Research and Development Department.
- Tencent (2025) WeKnora: LLM-Powered Document Understanding & Retrieval Framework. GitHub repository. Available at: <https://github.com/Tencent/WeKnora> (Accessed: 16 April 2026).
- Thakur, N., Reimers, N., Rücklé, A., Srivastava, A. and Gurevych, I. (2021) 'BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models', 35th Conference on Neural Information Processing Systems (NeurIPS 2021).
- Tunstall, L., von Werra, L. and Wolf, T. (2022) *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. Sebastopol, CA: O'Reilly Media, Inc.
- United Nations (2015) Transforming our world: the 2030 Agenda for Sustainable Development. Available at: <https://sdgs.un.org/2030agenda> (Accessed: 7 December 2025).
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. (2017) 'Attention is All you Need', *Advances in Neural Information Processing Systems*, 30.
- Wilcoxon, F. (1945) 'Individual comparisons by ranking methods', **Biometrics Bulletin**, 1(6), pp. 80–83.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J. and Han, S. (2024) 'SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models', arXiv preprint arXiv:2211.10438. Available at: <https://arxiv.org/abs/2211.10438> (Accessed: 13 April 2026).
- Xu, C., Gao, L., Miao, Y. and Zheng, X. (2025) 'Distributed Retrieval-Augmented Generation', arXiv preprint arXiv:2505.00443. doi: 10.48550/arXiv.2505.00443.
- Yao, Z., Dong, Z., Zheng, Z., Gholami, A., Yu, J., Tan, E., Wang, L. and Huang, Q. (2021) 'HAWQ-V3: Dyadic Neural Network Quantization', arXiv preprint arXiv:2011.10680. Available at: <https://arxiv.org/abs/2011.10680> (Accessed: 13 April 2026).
- Zylon-ai (2025) PrivateGPT: A Private, Offline, and Local-First Document Intelligence Platform. GitHub repository. Available at: <https://github.com/zylon-ai/private-gpt> (Accessed: 16 April 2026).